

Introduction

This is not an assessed exercise. You will be given assistance with the tools and with coding problems. Feedback and assistance will be provided over the course of the lab session. Sample solutions will be discussed, and further feedback will be provided, as appropriate.

Background

In the lectures, we introduced the concept of deadlock, and how it was shown to be different to successful termination. We discussed how we may show processes to be deadlock free or otherwise. In this lab exercise you will use the model checker FDR to construct and analyse your own model of the dining philosophers example, and investigate deadlock properties.

Task 1

In the lectures, we presented the dining philosophers problem. This problem consisted of five philosophers, and five forks. A simple model of a philosopher and a fork was presented, and this was used to construct a model of a college using parallel composition.

In the first part of this exercise, you are required to implement the model of the college in FDR. Your college should use the philosophers and the forks with exactly the same behaviours as presented in the seminar – you may not modify their behaviour in any way!

You should implement a suitable assertion that enables you to investigate the behaviour of your model – specifically with regards to deadlock. This assertion, when run, should highlight whether or not a deadlock exists and provide a suitable counter-example.

Task 2

In the second and third parts of this exercise, we are going to save the philosophers from their unsavoury plight!

Many solutions to solving the inherent deadlock in the dining philosophers have been proposed. One of the most appropriate is for the college to employ a butler. Philosophers continue life as they always have – but the butler guards the entrance to the dining room. The butler is aware that a deadlock problem may occur when all philosophers are allowed to enter the dining room at the same time, and has been instructed to make sure this never happens – essentially he ensures there are never too many philosophers in the dining room at any one time. Of course, the butler is also instructed not to be overly cautious also – and he should allow in as many philosophers as is safe to do so without risking deadlock.

You are required to design and model the butler such that he can prevent the deadlock from occurring. In doing so you may not alter the model of the philosophers or the forks (as presented in the seminar, and modelled in part 1). Your college will be amended – it will contain the philosophers and the forks as before, but also the butler, in parallel composition.

Submission deadline and procedure: not assessed.

You should check a suitable assertion that enables you to investigate the behaviour of your model – specifically with regards to deadlock. This assertion, when run, should highlight whether or not the deadlock persists. Of course, the aim is to prove that the deadlock has been removed!

Task 3

Life in the college continued happily for many years and the butler ensured that dining always ran smoothly and no philosopher starved to death. However one day, the butler announced his retirement. The college had to hastily come up with another solution as no more butlers were available. In considering the solution the college noted that although the philosophers continued to refuse to communicate with each other, they had mastered the art of rudimentary acknowledgements to the butler.

The college considered a number of solutions to the problem of replacing the butler, but they had no confidence that any of the proposals would solve the problem. Luckily, you can come to the rescue!

You are required to design a solution to ensure the college does not deadlock – but you may not use a butler. In doing so you may not alter the model of the forks (as forks cannot speak), but you may need to add some minimal behaviour into the model of a philosopher. Your college will be amended – it will contain the philosophers and the forks as before, and possibly any other new entity you design.

Common solutions include a waiter, whose permission is required by a philosopher in order to pick up a fork. The waiter gives permission to only one philosopher at a time until he has picked up both his forks – but putting a fork down is always allowed and does not require permission. Other solutions include considering recruiting an ambidextrous philosopher. You might even design your own novel, weird, and wonderful solution!

You should check a suitable assertion that enables you to investigate the behaviour of your model – specifically with regards to deadlock. This assertion, when run, should highlight whether or not the deadlock persists. Of course, the aim is to prove that the deadlock has been removed!

You should write a (short) report describing your (intended!) solutions to tasks 1, 2, and 3. The purpose of this report is to describe how your solution is expected to work, and explain how you have formulated this in your model.

Submission

You are not required to submit your work for this exercise as it is not an assessed exercise. However you should make sure you have completed this exercise, especially practising and understanding the use of communication and concurrency, as later laboratory work will build on this exercise.